PYTHON-ACTIVITY-BASED-LEARNING

CASE STUDY
1.EXAMINATION RESULT AND GRACE-MARK VALIDATION

Python Programming

1.**Problem Statement**

Examination Result and Grace-Mark Validation: Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. Objectives

Accept marks for multiple subjects.

Validate marks entered by the user.

Check subject-wise pass criteria.

Apply grace marks to eligible students.

Calculate total marks and display the final classification.

3. Assumptions

Each subject has a maximum of 100 marks.

A score of 35 or above is a pass.

Students scoring 32, 33, or 34 receive 3 grace marks.

Marks entered below 0 or above 100 are invalid.

Classification is based on total marks: 75% for Distinction, 60% for First Class, and 50% for Second Class.

These are sample rules for this case study and may differ from official college regulations.

4. Algorithm

Start the program.

Input the number of subjects and initialize total marks to zero.

Repeat for each subject and input the marks.

If marks are below 0 or above 100, display invalid marks.

If marks are 35 or above, display Pass and add marks to the total.

If marks are between 32 and 34, add 3 grace marks, display the result, and add the updated marks to the total.

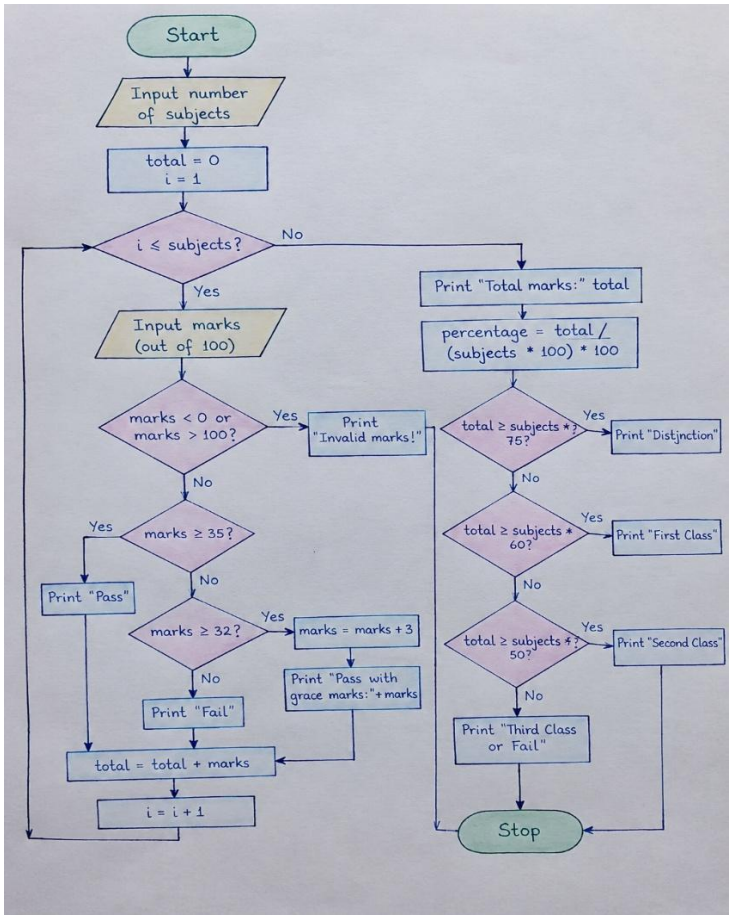
Otherwise, display Fail and add the marks to the total.

Display the total marks.

Compare total marks with the classification limits and display the classification.

Stop.

5. Flowchart



6. Pseudocode

START
INPUT number of subjects
SET total = 0

FOR each subject:
INPUT marks

IF marks < 0 OR marks > 100:
DISPLAY "Invalid marks!"

```
ELSE IF marks >= 35:
DISPLAY "Pass"
total = total + marks
ELSE IF marks >= 32:
marks = marks + 3
DISPLAY "Pass with grace marks"
total = total + marks
ELSE:
DISPLAY "Fail"
total = total + marks

DISPLAY total marks

IF total >= subjects * 75:
DISPLAY "Distinction"
ELSE IF total >= subjects * 60:
DISPLAY "First Class"
ELSE IF total >= subjects * 50:
DISPLAY "Second Class"
ELSE:
DISPLAY "Third Class or Fail"

STOP
```

7. Python Source Code

```
subjects = int(input("Enter number of subjects: "))
total = 0

for i in range(subjects):
  marks = int(input("Enter marks out of 100: "))

  if marks < 0 or marks > 100:
    print("Invalid marks!")
  elif marks >= 35:
    print("Pass")
    total = total + marks
  elif marks >= 32:
    marks = marks + 3
    if marks >= 35:
      print("Pass with grace marks:", marks)
      total = total + marks
```

```
    else:
        print("Fail")
        total = total + marks

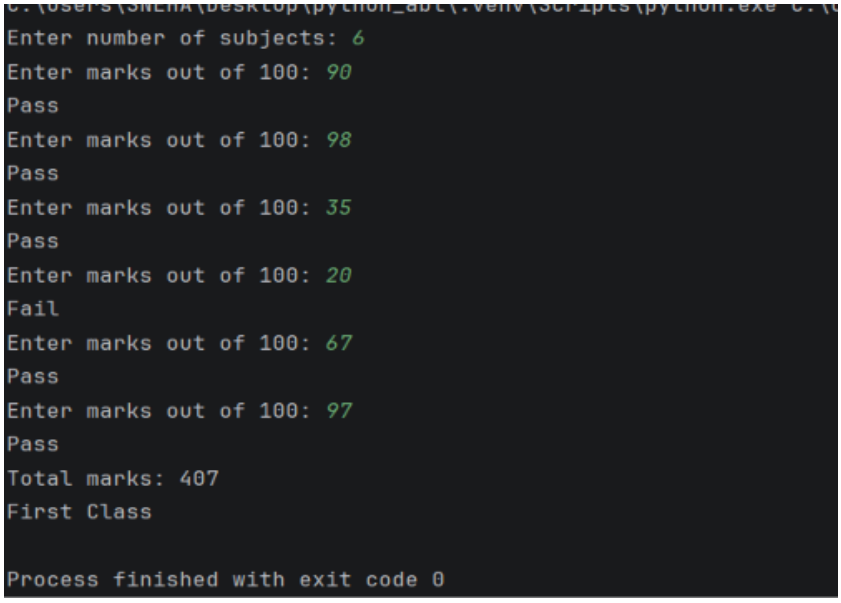
print("Total marks:", total)

if total >= subjects * 75:
    print("Distinction")
elif total >= subjects * 60:
    print("First Class")
elif total >= subjects * 50:
    print("Second Class")
else:
    print("Third Class or Fail")
```

8. Test Cases

Test Case	Input Marks	Expected Output	Purpose
1	80, 90, 85	Total = 255; Distinction	High marks
2	32, 60, 70	32 becomes 35 with grace; total = 165; Second Class	Grace-mark case
3	30, 60, 70	Fail for the subject; total = 160; Second Class by total	Below grace range
4	110, 70, 80	Invalid marks for 110; total = 150; Second Class	Invalid input
5	40, 45, 50	Total = 135; Third Class or Fail	Lower total

9. Sample Output Screenshot



10. Conclusion

The program accepts marks for multiple subjects, validates the entered marks, applies grace marks to eligible scores, calculates the total, and displays a classification based on the total marks. It demonstrates the use of variables, loops, arithmetic operators, and conditional statements in Python.

2.University Scholarship Allocation

1. Problem Statement

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Objectives

Accept applicant details.

Check eligibility using conditions.

Display Eligible, Waitlist, or Not Eligible.

3. Assumptions

Aggregate marks must be at least 85.

Minimum subject marks must be at least 70.

Attendance must be at least 75%.

Family income must not exceed ₹2,00,000.

These are sample rules.

4. Algorithm

Start and input marks, subject marks, attendance, and income.

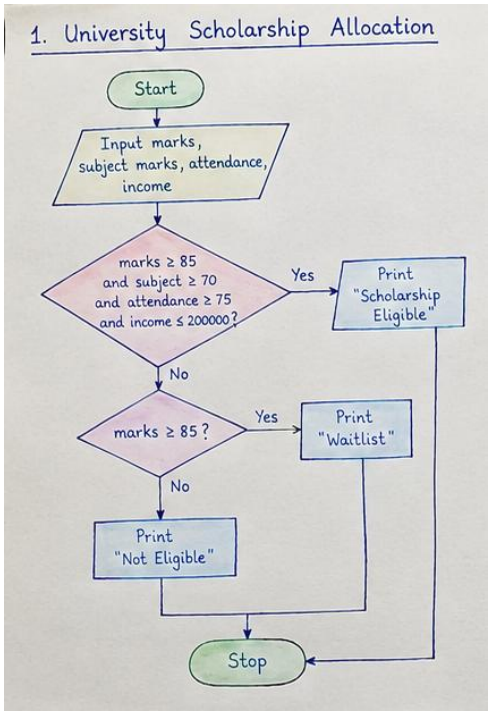
If all conditions are satisfied, display Scholarship Eligible.

Else if aggregate marks are at least 85, display Waitlist.

Otherwise display Not Eligible.

Stop.

5. Flowchart



6. Pseudocode

START
INPUT marks, subject, attendance, income
IF marks >= 85 AND subject >= 70 AND attendance >= 75 AND income <= 200000:
DISPLAY "Scholarship Eligible"
ELIF marks >= 85:
DISPLAY "Waitlist"
ELSE:

DISPLAY "Not Eligible"
STOP

7. Python Source Code

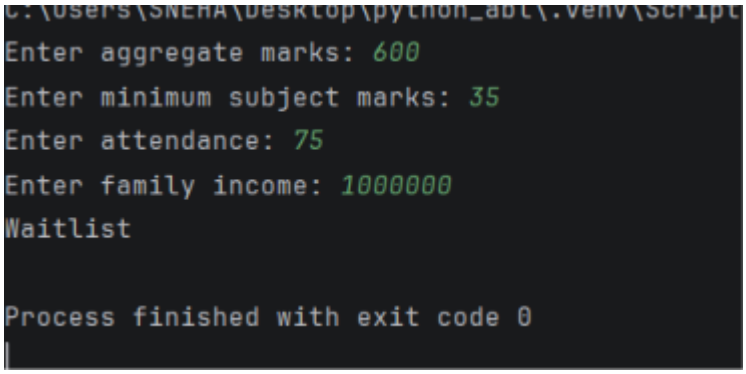
```
marks = int(input("Enter aggregate marks: "))
subject = int(input("Enter minimum subject marks: "))
attendance = int(input("Enter attendance: "))
income = int(input("Enter family income: "))

if marks >= 85 and subject >= 70 and attendance >= 75 and income <= 200000:
   print("Scholarship Eligible")
elif marks >= 85:
   print("Waitlist")
else:
   print("Not Eligible")
```

8. Test Cases

Test Case	Input	Expected Output
1	90, 80, 90, 150000	Scholarship Eligible
2	86, 65, 90, 150000	Waitlist
3	70, 80, 90, 150000	Not Eligible

9. Sample Output screenshot



10. Conclusion

The program checks scholarship eligibility from marks, attendance, and income and displays the applicant's status.

---

3. Progressive Electricity Billing

1. Problem Statement

Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Objectives

Calculate bills using slabs.

Add the fixed charge.

Store and compare monthly bills.

3. Assumptions

First 100 units cost ₹2 per unit.

Units 101–200 cost ₹3 per unit.

Units above 200 cost ₹5 per unit in this simplified example.

Fixed charge is ₹50.

An increase above 30% is unusual.

4. Algorithm

Input number of months.

For each month, input units and calculate the bill using slabs.

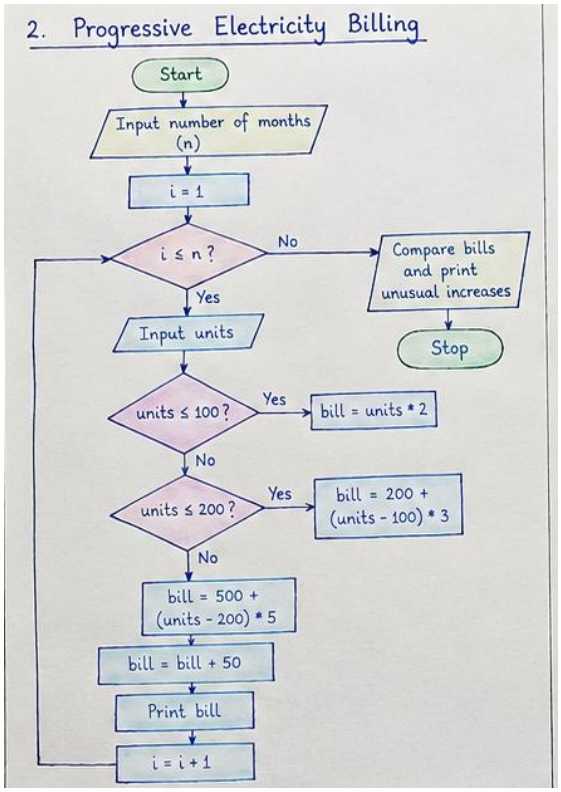
Add ₹50, store and display the bill.

Compare each bill with the previous bill.

Display a warning if the increase exceeds 30%.

Stop.

5. Flowchart



6. Pseudocode

START
INPUT n
CREATE empty bills list
FOR each month:
INPUT units
IF units <= 100: bill = units * 2
ELIF units <= 200: bill = 200 + (units - 100) * 3
ELSE: bill = 500 + (units - 200) * 5
bill = bill + 50
ADD bill to bills
DISPLAY bill
FOR each bill after the first:
IF current bill > previous bill * 1.3:
DISPLAY "Unusual increase!"
STOP

7. Python Source Code

n = int(input("Enter number of months: "))
bills = []

```
for i in range(n):
    units = int(input("Enter units: "))

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 200 + (units - 100) * 3
    else:
        bill = 500 + (units - 200) * 5

    bill = bill + 50
    bills.append(bill)
    print("Bill:", bill)

for i in range(1, len(bills)):
    if bills[i] > bills[i-1] * 1.3:
        print("Unusual increase!")
```

8. Test Cases

Test Case	Input	Expected Output
1	1 month, 80 units	Bill: 210
2	2 months, 50 then 100 units	Bills: 150 and 250; no warning
3	2 months, 50 then 150 units	Bills: 150 and 400; Unusual increase!

9. Sample Output screenshot



10. Conclusion

The program calculates simplified electricity bills, stores them in a list, and identifies unusually large increases.

---

4. Hospital Billing and Patient Classification

1. Problem Statement

Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Objectives

Input all charges and age.

Apply an age-based concession.

Classify the final bill.

3. Assumptions

Patients aged 60 or above receive a 10% concession.

A final bill above ₹5,000 is high-cost.

These are sample rules.

4. Algorithm

Input consultation, diagnostic, and treatment charges and age.

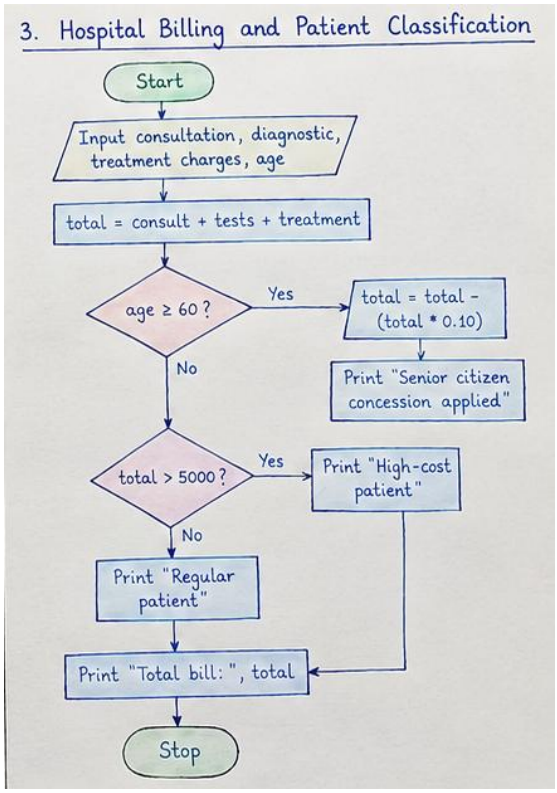
Add the charges.

If age is at least 60, apply a 10% concession.

If total exceeds ₹5,000, display High-cost patient; otherwise display Regular patient.

Display total and stop.

5. Flowchart



6. Pseudocode

START
INPUT consult, tests, treatment, age
total = consult + tests + treatment
IF age >= 60:
total = total - total * 0.10
DISPLAY concession message
IF total > 5000:
DISPLAY "High-cost patient"
ELSE:
DISPLAY "Regular patient"
DISPLAY total
STOP

7. Python Source Code

```
consult = int(input("Consultation charge: "))
tests = int(input("Diagnostic charges: "))
treatment = int(input("Treatment charges: "))
age = int(input("Enter patient age: "))

total = consult + tests + treatment
```

```
if age >= 60:
    total = total - total * 0.10
    print("Senior citizen concession applied")

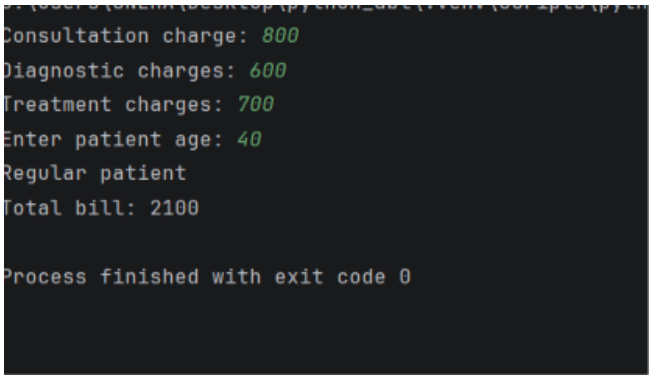
if total > 5000:
    print("High-cost patient")
else:
    print("Regular patient")

print("Total bill:", total)
```

8. Test Cases

Test Case	Input	Expected Output
1	500, 1000, 2000, age 40	Regular patient; total 3500
2	1000, 2000, 3000, age 65	Concession; High-cost patient; total 5400
3	500, 500, 1000, age 60	Concession; Regular patient; total 1800

9. Sample Output screenshot



10. Conclusion

The program adds hospital charges, applies a conditional concession, and classifies the final bill.

---

5. Employee Payroll and Deduction Analysis

1. Problem Statement

Employee Payroll and Deduction Analysis: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives

Calculate gross salary.

Choose a deduction rate using conditions.

Display gross salary, deduction, and net salary.

3. Assumptions

Gross salary up to ₹20,000 has 5% deduction.

Gross salary up to ₹50,000 after ₹20,000 has 10% deduction.

Gross salary above ₹50,000 has 15% deduction.

These are sample rates.

4. Algorithm

Input basic pay, allowances, overtime, and incentive.

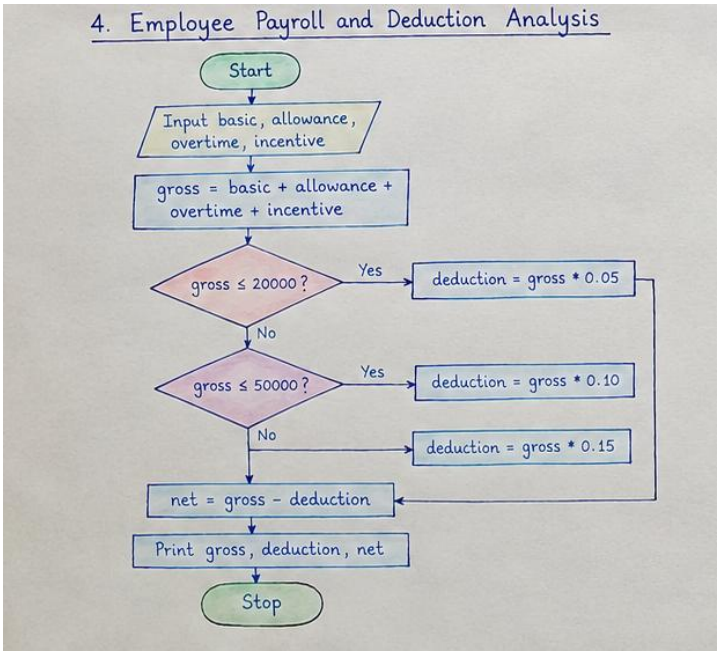
Add them to calculate gross salary.

Use if-elif-else to select the deduction rate.

Calculate net salary.

Display gross, deduction, and net salary. Stop.

5. Flowchart



6. Pseudocode

START
INPUT basic, allowance, overtime, incentive
gross = basic + allowance + overtime + incentive
IF gross <= 20000: deduction = gross * 0.05
ELIF gross <= 50000: deduction = gross * 0.10
ELSE: deduction = gross * 0.15
net = gross - deduction
DISPLAY gross, deduction, net
STOP

7. Python Source Code

```
basic = int(input("Enter basic salary: "))
allowance = int(input("Enter allowances: "))
overtime = int(input("Enter overtime pay: "))
incentive = int(input("Enter incentive: "))

gross = basic + allowance + overtime + incentive

if gross <= 20000:
   deduction = gross * 0.05
elif gross <= 50000:
   deduction = gross * 0.10
else:
   deduction = gross * 0.15
```

```
net = gross - deduction

print("Gross salary:", gross)
print("Deduction:", deduction)
print("Net salary:", net)
```

8. Test Cases

Test Case	Input	Expected Output
1	15000, 2000, 1000, 0	Gross 18000; deduction 900; net 17100
2	30000, 5000, 2000, 1000	Gross 38000; deduction 3800; net 34200
3	50000, 5000, 2000, 3000	Gross 60000; deduction 9000; net 51000

9. Sample Output



10. Conclusion

The program calculates gross salary and net salary using conditional deduction rates.

---

6. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Multi-Plan Telecommunications Cost Optimization: Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

2. Objectives

Input data usage and budget.

Calculate costs for three plans.

Choose the cheapest plan within budget.

3. Assumptions

Plan 1 costs ₹199 for 10 GB; extra usage costs ₹20 per GB.

Plan 2 costs ₹299 for 20 GB; extra usage costs ₹10 per GB.

Plan 3 costs ₹399.

All plans have sufficient validity; prices are illustrative.

4. Algorithm

Input usage and budget.

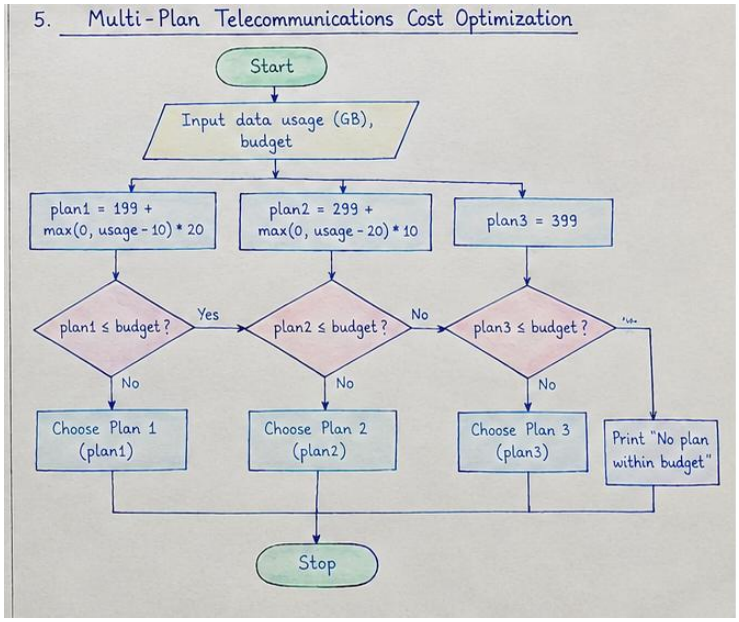
Calculate the cost of all three plans.

Check plans in order to find the cheapest plan within budget.

If none fits the budget, display No plan within budget.

Stop.

5. Flowchart



6. Pseudocode

START
INPUT usage, budget
plan1 = 199 + max(0, usage - 10) * 20
plan2 = 299 + max(0, usage - 20) * 10
plan3 = 399

IF plan1 <= budget AND plan1 <= plan2 AND plan1 <= plan3:
DISPLAY Plan 1
ELIF plan2 <= budget AND plan2 <= plan3:
DISPLAY Plan 2
ELIF plan3 <= budget:
DISPLAY Plan 3
ELSE:
DISPLAY "No plan within budget"
STOP

7. Python Source Code

```
usage = int(input("Enter data usage in GB: "))
budget = int(input("Enter budget: "))

plan1 = 199 + max(0, usage - 10) * 20
plan2 = 299 + max(0, usage - 20) * 10
plan3 = 399

if plan1 <= budget and plan1 <= plan2 and plan1 <= plan3:
   print("Choose Plan 1:", plan1)
elif plan2 <= budget and plan2 <= plan3:
   print("Choose Plan 2:", plan2)
elif plan3 <= budget:
   print("Choose Plan 3:", plan3)
else:
   print("No plan within budget")
```

8. Test Cases

Test Case	Input	Expected Output
1	Usage 5 GB, budget 250	Choose Plan 1: 199
2	Usage 15 GB, budget 350	Choose Plan 2: 299
3	Usage 50 GB, budget 200	No plan within budget

9. Sample Output screenshot

```
C:\Users\SNEHA\Desktop\python_abl\.venv\Scripts\python.
Enter data usage in GB: 3
Enter budget: 5000
Choose Plan 1: 199
```

10. Conclusion

The program calculates mobile plan costs and selects the lowest-cost option that satisfies the budget.

---

7. Railway Fare and Reservation Calculator

1. Problem Statement

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Objectives

Calculate fare; apply age concession; add reservation fee and calculate cancellation refund.

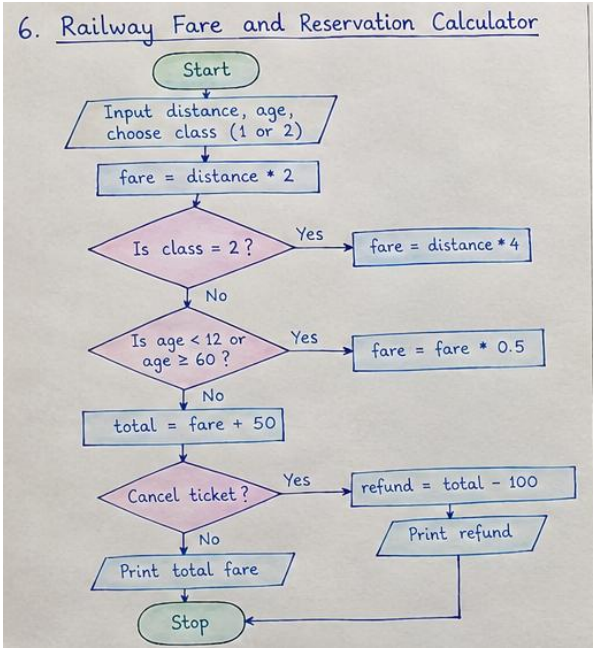
3. Assumptions

Sleeper fare = ₹2/km; AC fare = ₹4/km; children below 12 and senior citizens aged 60+ get 50% off the fare; reservation fee = ₹50; cancellation deduction = ₹100.

4. Algorithm

1. Input distance and age.
2. Ask the user to choose Sleeper or AC.
3. Calculate fare using the selected class.
4. Apply 50% concession if age < 12 or age >= 60.
5. Add ₹50 reservation fee and display total.
6. Ask if the ticket is cancelled; if yes, display total minus ₹100.
7. Stop.

5. Flowchart



6. Pseudocode

START
INPUT distance, age, class
fare ← distance × 2
IF class = 2 THEN fare ← distance × 4
IF age < 12 OR age >= 60 THEN fare ← fare × 0.5
total ← fare + 50
DISPLAY total
INPUT cancel
IF cancel = yes THEN DISPLAY total − 100
STOP

7. Python Source Code

```
distance = int(input("Enter distance: "))
age = int(input("Enter age: "))
print("1. Sleeper  2. AC")
cls = int(input("Choose class: "))

fare = distance * 2

if cls == 2:
   fare = distance * 4

if age < 12 or age >= 60:
   fare = fare * 0.5
```

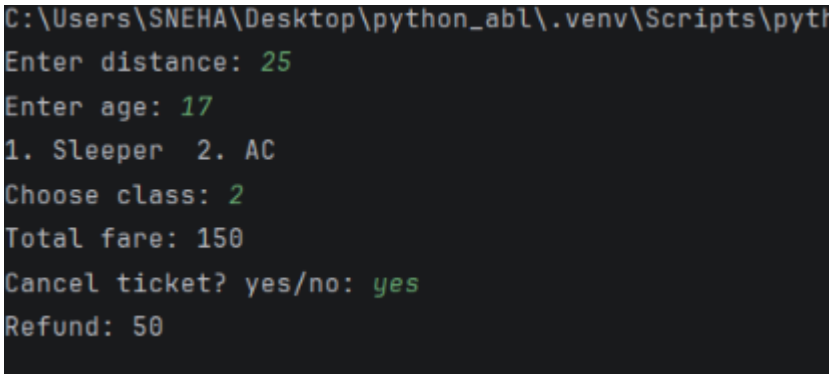
```
total = fare + 50
print("Total fare:", total)

cancel = input("Cancel ticket? yes/no: ")
if cancel == "yes":
   print("Refund:", total - 100)
```

8. Test Cases

Test Case	Input and Expected Output
1	1) distance=100, age=25, class=1, cancel=no → Total fare: 250.0
2	2) distance=100, age=65, class=2, cancel=yes → Total fare: 250.0; Refund: 150.0

9. Sample Output screenshot



10. Conclusion

The program calculates the ticket fare, applies the age concession, adds the reservation fee, and displays a refund when cancelled.

---

8. Retail Billing with Multiple Discount Rules

1. Problem Statement

Retail Billing with Multiple Discount Rules: Process multiple products, quantities, category specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Objectives

Calculate product amounts; apply the discount rule; add tax and calculate the final bill.

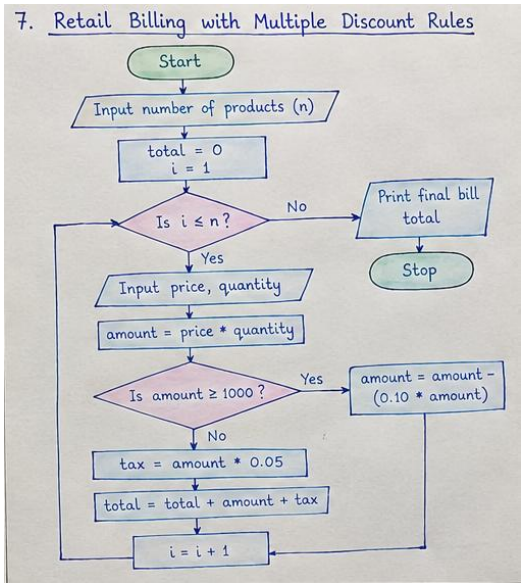
3. Assumptions

A 10% discount applies when price × quantity >= ₹1,000. Tax is 5% of the discounted amount. Prices and quantities are valid non-negative numbers.

4. Algorithm

1. Input number of products and set total to zero.
2. For each product, input price and quantity.
3. Calculate amount = price × quantity.
4. If amount >= 1000, subtract 10% discount.
5. Calculate 5% tax and add amount plus tax to total.
6. Display the final bill and stop.

5. Flowchart



6. Pseudocode

START
INPUT n
total ← 0
REPEAT n times:
INPUT price, quantity
amount ← price × quantity
IF amount >= 1000 THEN amount ← amount − 10% of amount
tax ← 5% of amount
total ← total + amount + tax
DISPLAY total
STOP

7. Python Source Code

```
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    price = int(input("Enter price: "))
    qty = int(input("Enter quantity: "))
    amount = price * qty

    if amount >= 1000:
        amount = amount - amount * 0.10

    tax = amount * 0.05
    total = total + amount + tax

print("Final bill:", total)
```

8. Test Cases

Test Case	Input and Expected Output
1	1) n=1, price=1000, qty=1 → Final bill: 945.0
2	2) n=2; (price=200, qty=2), (price=100, qty=1) → Final bill: 525.0

9. Sample Output screenshot



10. Conclusion

The program calculates bills for multiple products, applies the discount rule, adds tax, and displays the final amount.

---

9. Manufacturing Cost and Profitability Analysis

1. Problem Statement

Manufacturing Cost and Profitability Analysis: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. Objectives

Calculate total manufacturing cost; calculate profit; check the profit target.

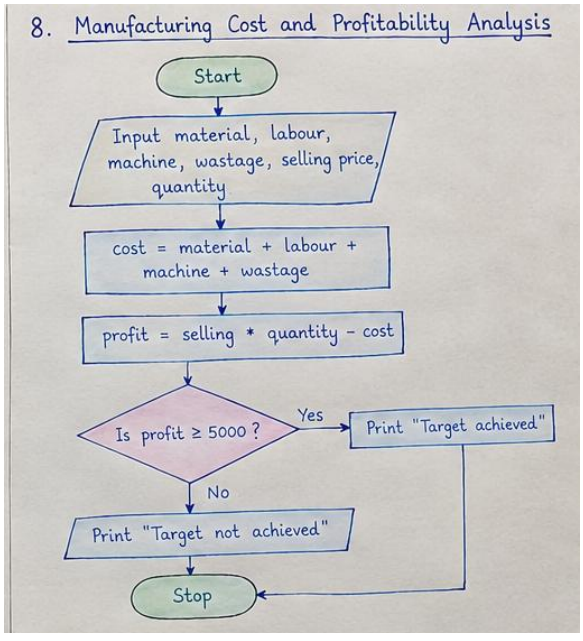
3. Assumptions

All costs and selling price are non-negative. Selling price is per unit. Profit target = ₹5,000.

4. Algorithm

1. Input material, labour, machine and wastage costs.
2. Input selling price per unit and quantity produced.
3. Add the costs to find total cost.
4. Calculate profit = selling price × quantity − total cost.
5. Display total cost and profit.
6. If profit >= 5000, display Target achieved; otherwise display Target not achieved.
7. Stop.

5. Flowchart



6. Pseudocode

START
INPUT material, labour, machine, wastage
INPUT selling, quantity

cost ← material + labour + machine + wastage
profit ← selling × quantity − cost
DISPLAY cost, profit
IF profit >= 5000 THEN DISPLAY Target achieved
ELSE DISPLAY Target not achieved
STOP

7. Python Source Code

```
material = int(input("Material cost: "))
labour = int(input("Labour cost: "))
machine = int(input("Machine cost: "))
wastage = int(input("Wastage cost: "))
selling = int(input("Selling price: "))
quantity = int(input("Quantity produced: "))

cost = material + labour + machine + wastage
profit = selling * quantity - cost

print("Total cost:", cost)
print("Profit:", profit)

if profit >= 5000:
   print("Target achieved")
else:
   print("Target not achieved")
```

8. Test Cases

Test Case	Input and Expected Output
1	1) costs=1000,1000,500,500; selling=200, quantity=50 → Total cost: 3000; Profit: 7000; Target achieved
2	2) same costs; selling=100, quantity=20 → Total cost: 3000; Profit: -1000; Target not achieved

9. Sample Output



10. Conclusion

The program calculates manufacturing expenses and profit, then checks whether the profit target has been reached.

---

10. Multi-Currency Travel Budget Analysis

1. Problem Statement

Multi-Currency Travel Budget Analysis: Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2. Objectives

Convert expenses to rupees; add a fixed fee; check if expenses are within budget.

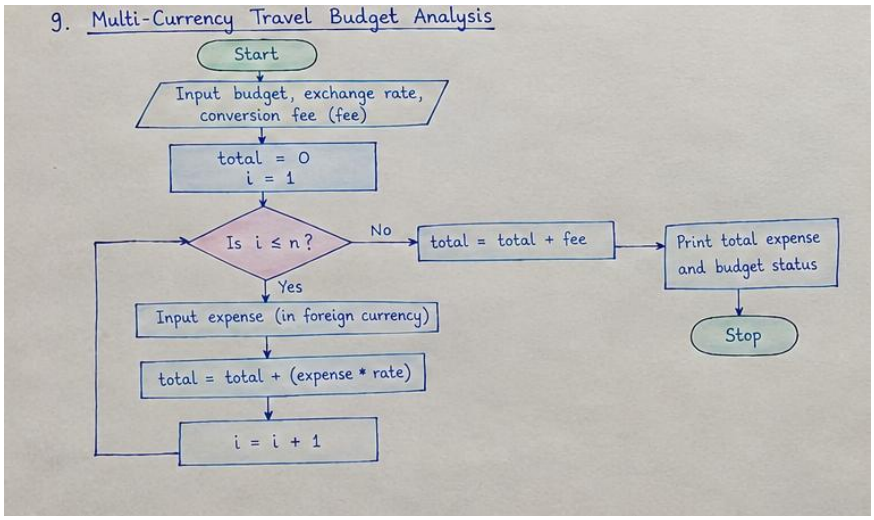
3. Assumptions

Exchange rate means rupees per one unit of foreign currency. Fixed fee = ₹100. Inputs are valid non-negative values.

4. Algorithm

1. Input budget and exchange rate.
2. Set fee = 100 and total = 0.
3. Input number of expenses.
4. For each expense, multiply it by the exchange rate and add it to total.
5. Add the ₹100 fee.
6. Display total expense.
7. Compare total with budget and display the result.
8. Stop.

5. Flowchart



6. Pseudocode

START
INPUT budget, rate
fee ← 100
total ← 0
INPUT n
REPEAT n times:
INPUT expense
total ← total + expense × rate
total ← total + fee
DISPLAY total
IF total <= budget THEN DISPLAY Within budget
ELSE DISPLAY Budget exceeded
STOP

7. Python Source Code

```
budget = int(input("Enter budget in rupees: "))
rate = float(input("Exchange rate: "))
fee = 100
total = 0

n = int(input("Number of expenses: "))

for i in range(n):
    expense = int(input("Enter expense in foreign currency: "))
    total = total + expense * rate
```

```
total = total + fee
print("Total expense in rupees:", total)

if total <= budget:
   print("Within budget")
else:
   print("Budget exceeded")
```

8. Test Cases

Test Case	Input and Expected Output
1	1) budget=5000, rate=80, expenses=20 and 30 → Total expense in rupees: 4100.0; Within budget
2	2) budget=3000, rate=80, expense=50 → Total expense in rupees: 4100.0; Budget exceeded

9. Sample Output screenshot



10. Conclusion

The program converts foreign expenses into rupees, includes the fixed fee, and checks whether the trip stays within budget.

DONE BY :

NAME : P. Sneha Anusha
ROLL NO. : 26001173
SEC : CSM-05